GenResearch-Servidor: memória, armazenamento, sincronização e desempenho de uma aplicação de RAG

Discente: Álex Felipe Santos Melo (202200025395) · **Professor:** Glauco de Figueiredo Carneiro
Repositório: https://github.com/alexfsmelo/SO_UFS_2026_2_Melo_Alex_ATV2

1. Introdução, problema e pergunta norteadora

Aplicações de IA generativa com recuperação aumentada (RAG) parecem, vistas de fora, um problema de modelo. Vistas de dentro, são um problema de sistema operacional: documentos são lidos do disco, fragmentados, convertidos em vetores e gravados em arquivos e bancos; consultas chegam em rajadas e esperam em filas; um índice de centenas de megabytes precisa estar acessível a várias threads; respostas recentes vão para um cache compartilhado; tudo é registrado em log. Cada uma dessas etapas disputa RAM, cache de páginas, espaço de endereçamento, disco e travas.

Este trabalho implementa e mede o **GenResearch-Servidor**, um serviço de RAG simplificado escrito em Python e desenhado para ser observável pelo sistema operacional. O foco não é a qualidade da resposta, e sim os custos que surgem durante a execução: quanto de memória cada forma de leitura consome, quando a concorrência ajuda e quando só cria fila, quanto o cache economiza e o que quebra quando um recurso compartilhado fica sem proteção.

Pergunta norteadora: como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e mecanismos de sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou de RAG?

Recorte experimental. Optou-se por corpus sintético em três tamanhos (S, M e L) mais um arquivo de vetores de 1 GB (XL) para a Parte E, embeddings simulados por *feature hashing* (vetores reais em float32, sem modelo neural) e geração simulada como espera de um servidor de inferência, calibrada em tokens por segundo. Essa escolha mantém o padrão de acesso à memória e ao disco de um RAG real e torna os experimentos repetíveis em minutos. A integração com Ollama está implementada (`--backend ollama`), mas não foi usada nas medições, porque o tempo de geração real (dezenas de segundos por resposta na AV1) esconderia os efeitos de memória e sincronização que se quer observar.

2. Ambiente (Parte A)

Tabela 2. Ambiente experimental (coletado por experiments/ambiente.sh: uname, os-release, lscpu, free, meminfo, lsblk, df, swapon, vmstat, iostat).

Item	Valor
Execução	Máquina virtual Linux leve (KVM, kernel 6.18.44-fc), isolada, sem GUI; systemd-detect-virt reporta contêiner (docker) e /proc/cpuinfo traz a flag hypervisor
Sistema operacional	Ubuntu 24.04.5 LTS, kernel 6.18.44, x86_64, PREEMPT_DYNAMIC
Processador	Intel Xeon @ 2,10 GHz, 2 vCPUs (1 thread por núcleo), L2 4 MiB, L3 260 MiB
Memória	7,8 GiB (MemTotal 8.222.320 kB); MemAvailable 7.717.048 kB na coleta; páginas de 4 KiB
Swap	Nenhum fixo (SwapTotal 0); arquivo de swap de 2 GB criado só durante o C5c; vm.swappiness = 60
Armazenamento	Disco virtio /dev/vda, 256 GB, ext4 (relatime, discard)
GPU	Não há; toda a computação em CPU
Software	Python 3.13.16, NumPy 2.5.3, SQLite 3.45.1, matplotlib 3.11, strace, sysstat (iostat, pidstat), vmstat
Modelo	Sem modelo neural nas medições: embeddings por hashing (256 dimensões) e geração simulada; Ollama opcional
Execução	20/10/2026; commit registrado no repositório; detalhes brutos em results/evidence/ambiente.txt

MemFree, MemAvailable, cache e swap. *MemFree* é a RAM que não está sendo usada para nada. *MemAvailable* é a estimativa do kernel de quanto pode ser entregue a novos processos sem recorrer a swap: soma a memória livre com a parte do cache de páginas e de estruturas do kernel que pode ser descartada. *Cached* é o cache de páginas, cópias de blocos de arquivos mantidas em RAM. *Swap* é a área em disco para onde vão páginas anônimas (heap, pilha, buffers do processo) quando falta RAM. Na coleta, MemFree e MemAvailable diferem em cerca de 224 MB justamente por causa do cache.

Cache de páginas não é desperdício. Uma página de arquivo limpa em cache pode ser descartada a qualquer momento sem escrita em disco; mantê-la custa quase nada e evita uma leitura futura. Por isso uma máquina "com pouca memória livre" pode estar saudável: o que importa é MemAvailable e, sob pressão, a taxa de faltas de página

maiores e de swap. Para RAG isso é central: datasets, índices vetoriais, embeddings e checkpoints de modelo são arquivos grandes, lidos repetidamente. Se cabem no cache de páginas, a segunda carga é muito mais rápida que a primeira (o efeito de "aquecimento" da AV1, quando o primeiro TTFT foi bem maior que os seguintes); se não cabem, cada acesso volta ao disco. Com `mmap`, o arquivo é paginado sob demanda e o mesmo quadro físico serve ao cache e ao processo.

3. Arquitetura

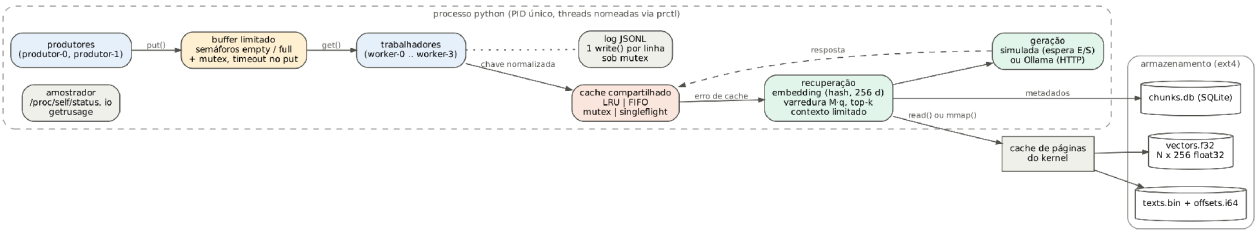


Figura 1. Arquitetura do GenResearch-Servidor: um processo, threads nomeadas no kernel (prctl), fila limitada, cache compartilhado, índice em arquivos e log.

Um único processo Python executa produtores e trabalhadores como threads. Os **produtores** geram consultas (carga determinística, semente fixa) e as colocam num **buffer limitado** implementado à mão com dois semáforos e um mutex. Os **trabalhadores** retiram requisições, calculam a chave normalizada da consulta e consultam o **cache compartilhado**. Em caso de erro de cache, fazem o embedding da consulta, varrem a matriz de vetores (produto M·q no NumPy, que libera o GIL), escolhem os top-k, montam um contexto limitado em tokens a partir de `texts.bin` e dos metadados no SQLite e chamam o **gerador**. Um **amostrador** lê /proc a cada 50 ms (fila, trabalhadores ocupados, RSS, VSZ, RssAnon, RssFile, swap, faltas de página, MemAvailable). Cada evento vai para um **log JSONL**. Cada execução grava resumo, uma linha por requisição, série temporal e log.

Por que threads. A etapa dominante é a geração, que num RAG real é uma chamada HTTP bloqueante a um servidor de inferência (como o Ollama da AV1): a thread dorme esperando E/S e libera o GIL. A recuperação é NumPy, que também libera o GIL na multiplicação. Threads permitem compartilhar o índice mapeado e o cache sem cópia nem serialização. Para mostrar o limite dessa escolha, o C3 inclui uma carga limitada por CPU em Python puro, na qual o GIL impede o paralelismo.

4. Dados, corpus, embeddings, índice e cache

Tabela 3. Corpus sintético e índice (semente 42). XL é uma matriz aleatória de 1 GiB usada só na Parte E.

Corpus	Docs	Trechos	Texto (MB)	Vetores (MB)	texts.bin (MB)	SQLite (MB)	Ingestão (s)	Trechos/s	Pico RSS (MB)
S	500	2.696	0,8	2,6	0,9	0,1	0,12	22.693	43
M	5.000	27.046	8,4	26,4	9,4	0,9	0,84	32.075	53
L	40.000	217.353	67,2	212,3	75,6	7,6	6,84	31.756	70
XL	-	1.048.576	-	1.024	-	-	-	-	-

Corpus. Documentos em português gerados por `corpus.py`, cada um de um entre dez tópicos de SO e IA (memória virtual, escalonamento, sistemas de arquivos, sincronização, embeddings, RAG, cache, armazenamento, inferência de LLM e GPU), com 8 a 40 frases que misturam termos do tópico e palavras comuns. Não há dados pessoais.

Fragmentação e embeddings. Trechos de até 400 caracteres com sobreposição de 50, cortados em limite de palavra. Cada token (sem stopwords) é mapeado por CRC32 para uma das 256 dimensões e um sinal; o vetor é normalizado, de modo que o produto escalar é a similaridade de cosseno. A ingestão processa trechos em lotes (4.096 por padrão; variado no C5b).

Índice. `vectors.f32` guarda a matriz N×256 em float32 contígua, sem cabeçalho, o que permite tanto `read()` do arquivo inteiro quanto `mmap()`; `texts.bin` e `offsets.i64` dão acesso direto ao texto de cada trecho; `chunks.db` (SQLite, WAL) guarda documentos e trechos. O carregamento valida o tamanho esperado de `vectors.f32` a partir de `meta.json` e levanta `IndiceIndisponivel` se o arquivo faltar ou estiver truncado.

Cache de respostas. Chave = conjunto ordenado dos tokens da consulta sem stopwords, de modo que mudanças de ordem, caixa e pontuação colidem. Capacidade configurável (0 desliga), políticas LRU e FIFO, três modos de concorrência (`none`, `lock`, `singleflight`) e contadores de acertos, erros, remoções, cálculos redundantes e tempo economizado (soma do custo original das entradas reutilizadas).

5. Processos, threads, filas e recursos compartilhados

Cada execução é um processo com, no pico, uma thread principal, um amostrador, dois produtores e de um a quatro trabalhadores. Os nomes são gravados no kernel com `prctl(PR_SET_NAME)`, o que os torna visíveis em `ps -L` e `/proc/PID/task/*/comm` (evidência em `results/evidence/processo_vivo.txt`, Anexo A). Na amostra com quatro trabalhadores, eles estavam em `hrtimer_nanosleep` (esperando o servidor de inferência simulado) e os produtores em `futex_do_wait` (bloqueados no semáforo `empty` da fila cheia); apareceu também uma thread sem nome com 92% de CPU, o pool do OpenBLAS que o NumPy usa na multiplicação M·q, um lembrete de que bibliotecas nativas criam threads que o código Python não declara. A Tabela 4 lista os recursos compartilhados e a proteção de cada um.

Tabela 4. Recursos compartilhados e sincronização.

Recurso compartilhado	Quem acessa	Risco sem proteção	Mecanismo adotado
Buffer de requisições (deque)	produtores e trabalhadores	perda ou duplicação de itens, consumidor lendo fila vazia	semáforos empty/full + mutex; timeout em put/get
Cache de respostas (OrderedDict)	trabalhadores	atualização perdida nos contadores, cálculo duplicado, exceção ao remover	mutex; singleflight (Event por chave)
Arquivo de log	todas as threads	linhas intercaladas e JSON inválido	mutex + um único write() por linha (O_APPEND)
Lista de resultados e contadores	trabalhadores, produtores	atualização perdida	mutex
Matriz de vetores e textos	trabalhadores (só leitura)	nenhum (imutável após a carga)	sem trava; mmap compartilhado
Conexão SQLite	trabalhadores	uso de objeto sqlite3 entre threads	uma conexão por thread (threading.local), modo somente leitura
Memo de tokens do embedder	trabalhadores	corrida benigna (mesmo valor gravado)	sem trava, documentado

6. Estratégia de sincronização

Produtores e consumidores (Opção 1). `BoundedBuffer` usa `empty` (vagas, inicia na capacidade), `full` (itens, inicia em zero) e `mutex` (protege o deque e as estatísticas). `put()` faz `empty.acquire(timeout)`, insere sob o mutex e libera `full`; `get()` faz o inverso. O timeout transforma fila cheia em rejeição controlada, com retentativa e recuo exponencial no produtor, em vez de bloqueio indefinido. O encerramento usa uma pílula `PARAR` por trabalhador, inserida depois que todos os produtores terminam.

Cache compartilhado (Opção 2). No modo `none` a leitura e a gravação não têm exclusão mútua, e uma pausa `time.sleep(0)` entre ler e escrever cada contador amplia a janela de corrida para torná-la reproduzível. No modo `lock` o mutex protege a estrutura e os contadores, mas o cálculo da resposta fica fora da região crítica para não serializar a geração; por isso dois trabalhadores que erram a mesma chave ao mesmo tempo ainda calculam duas vezes. O modo `singleflight` resolve isso: o primeiro registra um `threading.Event` para a chave e calcula; os demais esperam o evento e então leem o resultado. O custo é espera, e não CPU.

Log. A linha JSON é montada inteira e gravada com um único `os.write()` sob mutex num descritor aberto com O_APPEND. A falha controlada grava a mesma linha em fragmentos de 16 bytes sem mutex.

Deadlock e starvation. O risco de espera circular entre a trava do índice e a do cache é demonstrado num processo isolado, com três versões: ingênua (ordens opostas, encerrada pelo processo pai após 3 s), com `acquire(timeout)` e recuo aleatório (detecção e recuperação) e com ordem global de aquisição (prevenção). Starvation é mostrada com uma trava leitores/escritor com preferência a leitores, comparada à versão com preferência a escritores.

7. Metodologia

Cada configuração foi executada **três vezes**, sempre em processo novo, para que RSS, faltas de página e contadores de E/S não herdem estado. A máquina não tinha outras cargas relevantes durante as medições, e os experimentos rodaram em sequência. Todos os parâmetros, horários e resultados brutos estão nos CSVs de `results/raw/`; as tabelas e figuras deste relatório são geradas por `experiments/analise.py` a partir deles (média de três repetições; barras de erro = desvio-padrão). Os fatores variados foram: número de trabalhadores (1, 2 e 4), tamanho do corpus (S, M, L, XL), capacidade da fila (2, 4, 16, 64), tamanho e política do cache (0, 8, 64; LRU, FIFO), tamanho do contexto

(top-k 4 e 64), lote de embeddings (256, 4.096, 65.536), modo de leitura (read, mmap, madvise) e limite de memória (cgroup).

Tabela 5. Configurações experimentais (três repetições cada).

Config.	O que varia	Fixo	Métricas principais
C1/C2	read x mmap; S, M, L, XL; varredura x acesso esparso (256 linhas); cache frio (drop_caches) x quente; madvise	processo novo por medida	tempo de carga e 1º acesso, RSS anônima e de arquivo, VSZ, faltas de página, bytes lidos
C3	1, 2, 4 trabalhadores; fila 4 e 64; carga de E/S x CPU	2 produtores, 120 req., corpus M, sem cache	vazão, latência média e p95, espera na fila, tamanho máx. da fila, utilização
C4	carga repetida, paráfrase, mista e distinta; cache 0/8/64; LRU/FIFO; lock/singleflight	2 trab., 150 req., chegada 10 ms	taxa de acerto, remoções, recuperações evitadas, tempo economizado
C5	corpus S/M/L x read/mmap x contexto; lote; cgroup 64 a 384 MB, com e sem swap	2 trab., cache desligado	RSS, VSZ, faltas maiores, bytes lidos, swap, latência, OOM
C6	arquivo ausente, índice ausente/truncado, fila saturada, limite de memória, falha de serviço externo	corpus S, M ou L	código de saída, erros, retentativas, descartes, recuperação
D	sem e com proteção; servidor e micro-testes; deadlock; starvation	8 threads nos micro-testes	atualizações perdidas, cálculos redundantes, linhas corrompidas, bloqueios

Ferramentas. `/proc/self/status` (VmRSS, VmSize, VmHWM, RssAnon, RssFile, VmSwap, Threads), `/proc/self/io` (rchar, read_bytes, syscr), `getrusage` (faltas menores e maiores, trocas de contexto), `mincore` via ctypes (fração do arquivo no cache de páginas), `strace -c` e `strace -e`, `ps -L`, `pidstat -t`, `vmstat 1`, `iostat -xz 1` e os contadores do cgroup (`memory.failcnt`, `memory.oom_control`). Cache frio = `sync; echo 3 > /proc/sys/vm/drop_caches` mais `posix_fadvise(DONTNEED)` no arquivo. Os limites de memória foram aplicados por cgroup filho do próprio processo de experimento, de modo que nada fora dele foi afetado.

Geração simulada. Espera = tokens de contexto / 20.000 tok/s + 16 tokens de saída / 400 tok/s, cerca de 65 ms por resposta com 500 tokens de contexto. A carga de CPU do C3 substitui a espera por cerca de 40 ms de trabalho fixo em Python puro (calibrado no início do processo).

8. Resultados

8.1 C1 e C2: leitura convencional x mmap (Parte E)

Tabela 6. C1 x C2 para L (212 MB) e XL (1 GiB): abertura + carga + primeiro acesso; média de 3 repetições. S e M em results/tables/t_c1c2.md.

Arquivo	Acesso	Modo	Frio (ms)	Quente (ms)	ΔRSS (MB)	RssAnon	RssFile	Faltas menores	Faltas maiores (frio)	Lido do disco (MB, frio)
L	varredura	read	124,1	116,6	218	228	18	55.015	19	214
L	varredura	mmap	101,6	15,5	218	16	230	3.951	848	214
L	esparso	read	127,7	108,6	218	228	18	54.856	22	214
L	esparso	mmap	24,8	1,2	21	16	33	525	264	32
XL	varredura	read	632,9	621,8	1.030	1.040	18	263.130	19	1.026
XL	varredura	mmap	509,5	80,5	1.030	16	1.042	15.869	3.799	1.026
XL	esparso	read	666,9	563,0	1.030	1.040	18	262.672	22	1.026
XL	esparso	mmap	29,4	1,1	22	16	33	526	275	33

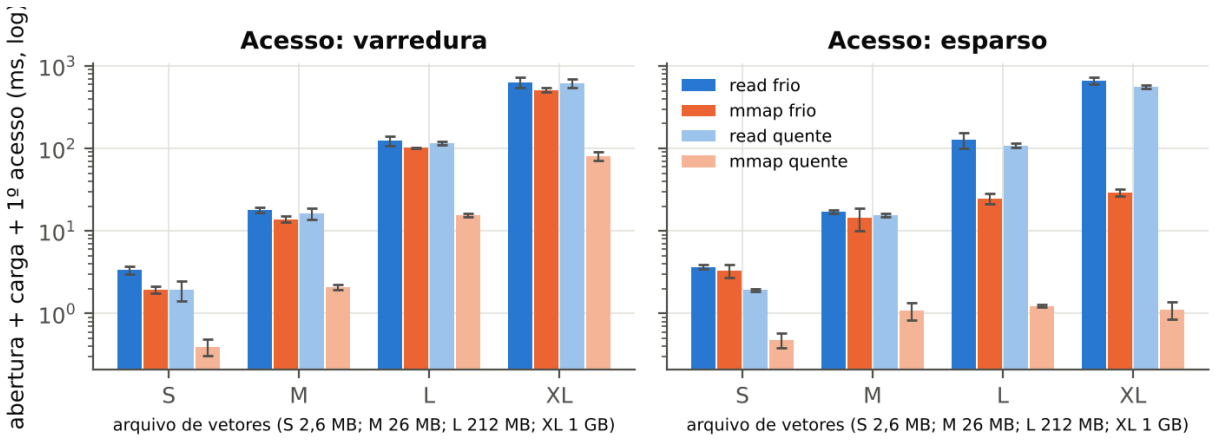


Figura 2. Tempo até o primeiro resultado por tamanho de arquivo (escala log). Cache frio = drop_caches antes de cada processo.

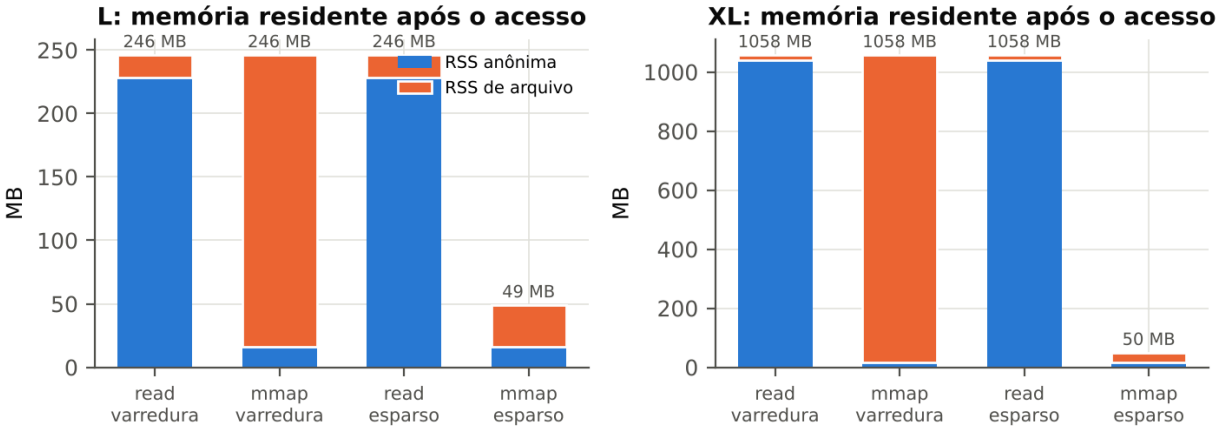


Figura 3. Memória residente após o acesso, separada em anônima (heap do processo) e de arquivo (páginas mapeadas do cache de páginas). Cache quente.

Leitura convencional. Com `read()` o processo aloca um buffer do tamanho do arquivo e o kernel copia nele cada página. O RSS cresce exatamente o tamanho do arquivo (ΔRSS de 1.030 MB no XL), quase tudo memória anônima, e a carga gera 262.146 faltas de página menores: uma por página de 4 KiB do buffer novo, preenchida com zeros antes da cópia. Por isso o `read()` com cache quente continua caro (622 ms contra 633 ms frio no XL): o disco sai do caminho, mas a alocação e a cópia ficam. O strace confirma: depois do `openat`, uma única chamada `read(fd, buf, 222569473)` devolve os 212 MB do L de uma vez; no modo mmap, a mesma abertura é seguida de um único `mmap(NULL, 222569472, PROT_READ, MAP_SHARED, fd, 0)` e de nenhum `read()` (Anexo A).

mmap. Com `mmap()` a carga é instantânea (microssegundos) e só reserva espaço de endereçamento: a VSZ do processo cresce aproximadamente o tamanho do arquivo (ΔVSZ de 1.064 MB no XL, medido após o acesso), sem que nenhuma página seja lida nesse momento. As páginas entram na primeira vez que são tocadas, como faltas de página: maiores quando vêm do disco (3.799 na varredura fria do XL, graças ao *readahead*, que traz blocos de páginas por falta) e menores quando já estão no cache (15.866 na varredura quente). Na varredura, o RSS final é o mesmo do `read()`, mas é RSS **de arquivo**: páginas limpas que pertencem ao cache de páginas e que o kernel pode descartar sob pressão sem swap. Com cache quente, o mmap levou 80 ms contra 622 ms do read no XL, porque não há cópia.

Acesso esparso é onde o mmap domina. Tocando 256 linhas aleatórias, o mmap ocupou 22 MB de RSS no XL contra 1.030 MB do read, e respondeu em 29,4 ms com cache frio contra 667 ms. Mesmo assim leu 33 MB do disco para 256 KB úteis: cada falta maior dispara readahead. No L, com cache frio, o acesso esparso deixou 14% do arquivo no cache de páginas (medido com mincore).

Tabela 7. Padrão de leitura informado ao kernel: mmap do L, varredura completa, cache frio (3 repetições).

madvise	1º acesso (ms)	Faltas maiores	Faltas menores	Lido do disco (MB)
padrão	101,6	848	3.950	214
WILLNEED	100,6	758	3.822	214
SEQUENTIAL	123,4	870	3.224	214

madvise	1º acesso (ms)	Faltas maiores	Faltas menores	Lido do disco (MB)
RANDOM	1.241,8	54.358	674	214

Padrão de leitura. `MADV_RANDOM` desliga o readahead: a mesma varredura passou a gerar 54.358 faltas maiores (uma por página) e levou 1.242 ms, contra 102 ms no padrão. `MADV_WILLNEED` dispara a leitura antecipada logo no `madvise` e reduziu as faltas maiores para 758. O total lido do disco é o mesmo nos quatro casos; o que muda é quantas vezes a thread para e espera o disco.

8.2 C3: concorrência e fila

Tabela 8. C3 com 2 produtores e 120 requisições em rajada, cache desligado, corpus M (média de 3). Perdidas e duplicadas: 0 em todas as execuções.

Carga	Fila	Trab.	Vazão (req/s)	Lat. média (ms)	p95 (ms)	Serviço (ms)	Espera fila (ms)	Fila média	Utilização	Threads
E/S	4	1	15,0	451	481	66,3	385	3,5	100%	6
E/S	4	2	30,2	255	269	65,9	190	3,4	99%	7
E/S	4	4	59,4	160	192	66,2	94	3,4	98%	9
E/S	64	1	15,1	3.185	4.376	66,1	3.119	46,8	100%	5
E/S	64	2	29,4	1.665	2.316	67,5	1.597	46,9	99%	6
E/S	64	4	59,7	836	1.163	66,0	770	47,1	99%	8
CPU	4	1	26,4	264	339	38,7	225	3,5	99%	6
CPU	4	2	22,7	341	419	85,5	256	3,4	95%	7
CPU	4	4	21,2	432	602	173,5	258	3,2	92%	9
CPU	64	1	22,5	2.171	3.078	44,3	2.127	46,8	100%	6
CPU	64	2	21,2	2.301	3.301	90,4	2.211	46,8	95%	6
CPU	64	4	21,7	2.311	3.267	167,9	2.143	47,0	91%	8

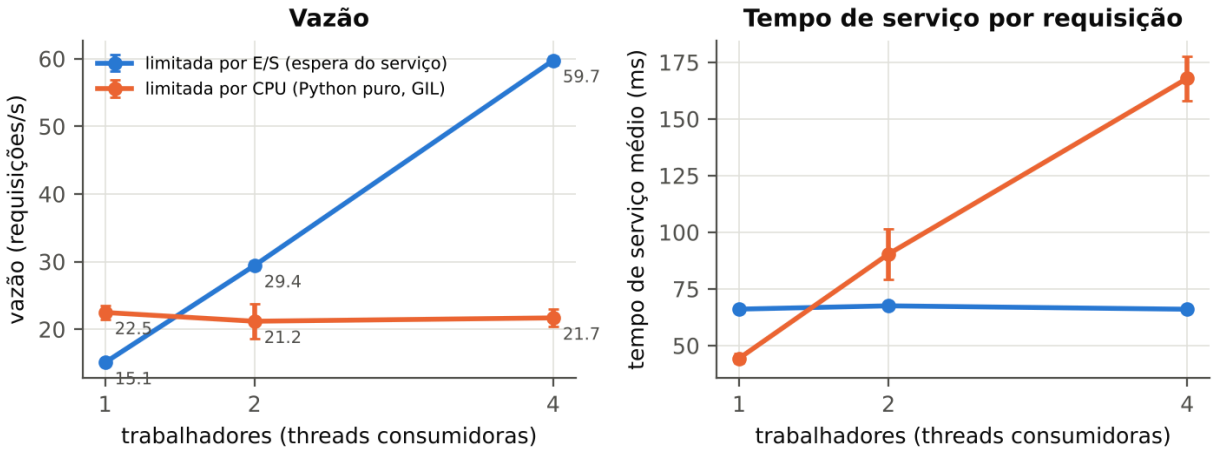


Figura 4. Vazão e tempo de serviço por número de trabalhadores (fila 64). Barras de erro: desvio-padrão de 3 repetições.

Carga limitada por E/S. A vazão cresceu de 15,1 para 29,4 e 59,7 req/s com 1, 2 e 4 trabalhadores (3,96x), com tempo de serviço estável em cerca de 66 ms. Mesmo com só 2 vCPUs, quatro threads escalam porque passam a maior parte do tempo dormindo na espera do "servidor de inferência", e a parte de CPU (NumPy) libera o GIL.

Carga limitada por CPU. Com trabalho fixo em Python puro, a vazão foi 22,5, 21,2 e 21,7 req/s, e o tempo de serviço de cada requisição subiu de 44 para 168 ms: as threads disputam o GIL, então mais trabalhadores só repartem o mesmo núcleo efetivo e aumentam as trocas de contexto involuntárias (1.043 para 4.081). Para essa carga o mecanismo certo seria multiprocessamento.

Fila. Como os produtores enviam em rajada, a fila fica quase sempre cheia (média 46,8 de 64). Aumentar a capacidade de 4 para 64 não mudou a vazão (15,0 x 15,1 req/s com 1 trabalhador), mas multiplicou a espera na fila (385 para 3.119 ms) e o p95. Fila maior só troca bloqueio do produtor por latência do cliente; com fila 4, o tempo de espera aparece como produtor bloqueado no semáforo `empty`. A contenção começa quando a taxa de chegada supera a capacidade de serviço: a partir daí toda folga vira fila (série temporal na Figura A1, Anexo A).

8.3 C4: cache compartilhado

Tabela 9. C4: taxa de acerto e tempo médio de serviço por carga e configuração (150 requisições, 2 trabalhadores, média de 3). SF = singleflight.

Carga	sem cache	8 LRU	8 FIFO	64 LRU	64 FIFO	64 LRU SF
repetida	0% · 65 ms	55% · 30 ms	49% · 34 ms	84% · 11 ms	84% · 11 ms	85% · 10 ms
paráfrase	0% · 66 ms	23% · 51 ms	21% · 52 ms	62% · 25 ms	62% · 25 ms	63% · 25 ms
mista	0% · 66 ms	16% · 56 ms	13% · 57 ms	35% · 43 ms	33% · 45 ms	35% · 43 ms
distinta	0% · 66 ms	0% · 66 ms	0% · 66 ms	0% · 66 ms	0% · 66 ms	0% · 66 ms

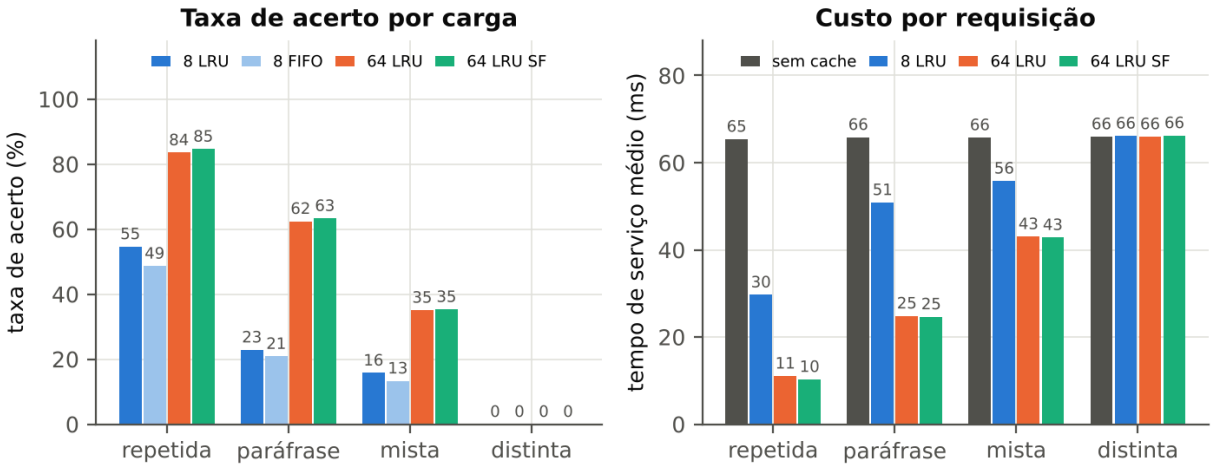


Figura 5. Taxa de acerto e custo por requisição. Consultas distintas nunca acertam; paráfrases acertam menos que repetições exatas.

Ganho. Com consultas repetidas (24 consultas sorteadas com pesos tipo Zipf), o cache de 64 entradas acertou 84% e reduziu as recuperações de 150 para 25, o tempo médio de serviço de 65 para 11 ms e a duração total de 4,92 para 1,09 s; o tempo economizado somado foi 8,7 s. Com consultas distintas a taxa foi 0% e o cache só gerou remoções (142 no cache de 8), sem custo mensurável no tempo de serviço.

Capacidade e política. Com 8 entradas, menores que o conjunto de 24 consultas, há substituições, e a política importa: LRU acertou 55% e FIFO 49%, porque o LRU mantém as consultas populares que FIFO descarta por idade. Com 64 entradas tudo cabe e as duas políticas empatam.

Paráfrases. A normalização (ordem, caixa, pontuação e stopwords) fez paráfrases acertarem 62%, abaixo das repetições exatas, porque prefixos como "qual o papel de" ou "quando usar" acrescentam palavras de conteúdo e mudam a chave. Um cache semântico (vizinho mais próximo do embedding da consulta com limiar) acertaria mais, com risco de devolver a resposta de uma pergunta diferente.

Concorrência no cache. No modo `lock`, consultas populares que chegam juntas ainda são calculadas mais de uma vez (1,7 cálculos redundantes em média); o singleflight zerou esse número (0) e levou a taxa de acerto a 85%.

8.4 C5: pressão de memória

Tabela 10. C5a: corpus, modo de leitura e contexto (top-k 4 e até 256 tokens; top-k 64 e até 4.096 tokens). 60 requisições, 2 trabalhadores, média de 3. Memória em MB.

Corpus	Modo	Contexto	Carga (ms)	RSS após carga	Pico RSS	RssAnon	RssFile	Faltas menores	Recup. (ms)	Geração (ms)	Trechos	Tokens
S	read	pequeno	3,4	40	42	23	19	1.179	0,8	51	3,2	220
S	read	grande	2,4	40	42	23	19	1.295	1,1	243	57,2	4.057
S	mmap	pequeno	0,1	36	42	19	22	325	0,5	51	3,2	220
S	mmap	grande	0,1	36	42	20	22	441	1,5	243	57,2	4.057
M	read	pequeno	21,1	72	76	57	19	12.830	1,8	51	3,4	218
M	read	grande	22,4	72	77	58	19	12.987	3,7	244	58,4	4.057
M	mmap	pequeno	0,1	36	74	21	53	4.240	2,0	51	3,4	218
M	mmap	grande	0,1	36	77	22	55	3.754	3,3	243	58,4	4.057
L	read	pequeno	173,7	326	336	317	19	82.378	20,5	51	3,6	210
L	read	grande	150,5	326	339	320	19	83.116	20,9	243	60,9	4.038
L	mmap	pequeno	0,1	36	324	26	296	9.772	14,3	51	3,6	210

Corpus	Modo	Contexto	Carga (ms)	RSS após carga	Pico RSS	RssAnon	RssFile	Faltas menores	Recup. (ms)	Geração (ms)	Trechos	Tokens
L	mmap	grande	0,2	36	337	27	308	27.657	26,4	243	60,9	4.038

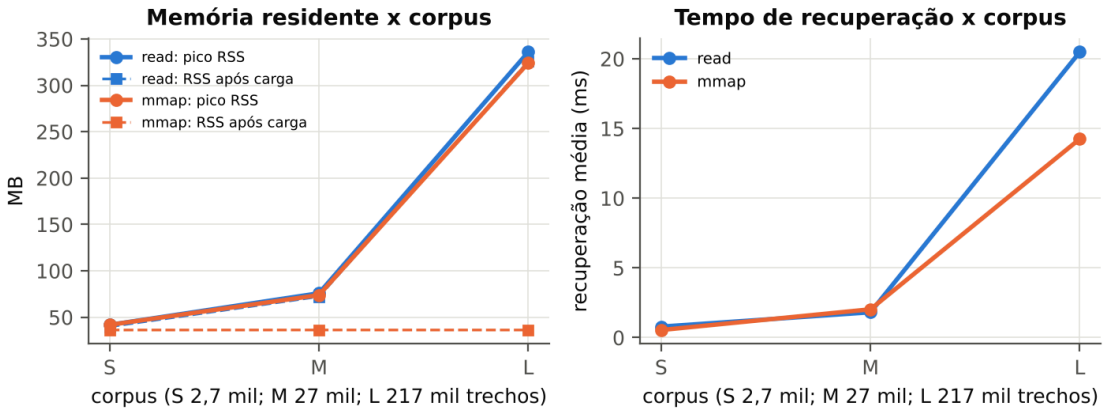


Figura 6. Memória residente e tempo de recuperação em função do corpus (contexto pequeno).

Tamanho do corpus. Com `read`, o RSS logo após carregar o índice acompanha o tamanho dos arquivos: 40 MB no S e 326 MB no L (vetores, textos e deslocamentos copiados para memória anônima), com 174 ms de carga e 82.378 faltas menores. Com `mmap`, a carga leva 0,12 ms e o RSS inicial é o do interpretador (36 MB); o pico chega a 324 MB porque a varredura toca a matriz inteira, mas 296 MB disso é RSS de arquivo. A recuperação cresce linearmente com o número de trechos (0,5 ms no S, 14,3 ms no L): a busca é uma varredura exata, e o custo é de largura de banda de memória, não de disco, já que o arquivo está em cache.

Contexto. Passar de top-k 4 para 64 trechos (≈4.038 tokens) aumentou a geração simulada de cerca de 51 para 243 ms, porque o custo de *prefill* cresce com os tokens de entrada, e multiplicou por 2,8 as faltas de página da execução no L com mmap (9.772 para 27.657), porque mais páginas de `texts.bin` são tocadas para montar o contexto. Num LLM real, o mesmo crescimento aparece no KV-cache, que ocupa memória proporcional ao contexto.

Lote. Na ingestão do corpus M, o pico de RSS foi 41 MB com lote de 256 trechos, 54 MB com 4.096 e 109 MB com 65.536, para tempos praticamente iguais (0,95, 0,94 e 0,84 s). Lotes maiores compram pouca vazão com muita memória: o mesmo compromisso do tamanho de lote em inferência e em geração de embeddings com GPU.

Tabela 11. C5c: corpus L (vetores 212 MB + textos 76 MB) sob limite de memória, 40 requisições, cache frio antes de cada execução. Médias só das execuções concluídas.

Modo	Limite (cgroup)	Swap	Concluídas	OOM	Recup. (ms)	Duração (s)	Faltas maiores	Lido do disco (MB)	Swap pico (MB)	failcnt
mmap	sem limite	não	3/3	0	30,3	1,94	704	248	0	-
mmap	160 MB	não	3/3	0	261,3	6,56	15.654	7.354	0	0
mmap	96 MB	não	3/3	0	299,1	7,30	20.444	8.511	0	0
mmap	64 MB	não	3/3	0	308,7	7,54	20.132	8.537	0	0
read	sem limite	não	3/3	0	20,9	1,71	22	294	0	-
read	384 MB	não	3/3	0	23,2	1,76	40	296	0	0
read	192 MB	não	0/3	3	-	-	-	-	-	0
read	256 MB	sim	3/3	0	49,8	2,35	6.570	442	96	59.604
read	192 MB	sim	2/3	1	661,4	14,77	170.154	4.150	159	631.845

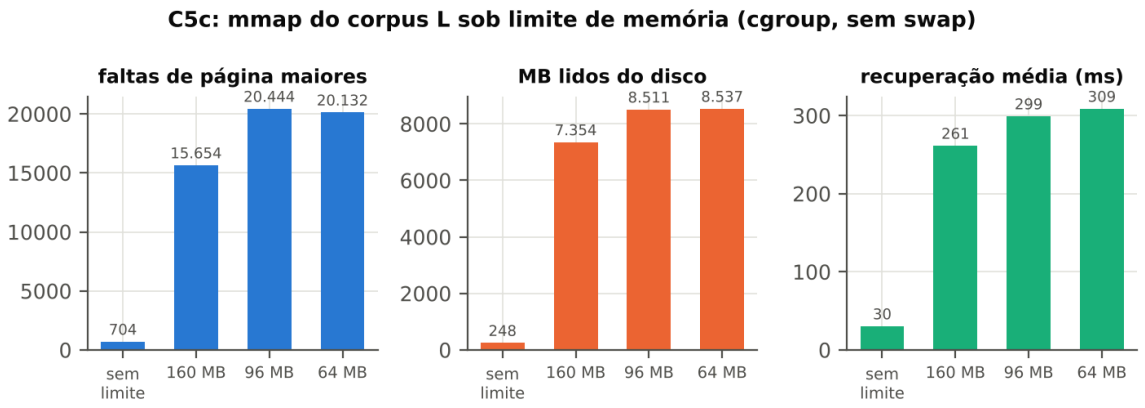


Figura 7. mmap do corpus L com limites de memória decrescentes: abaixo do tamanho do arquivo, as páginas de arquivo são descartadas e relidas a cada varredura.

mmap sob limite: degradação. Sem limite, o mmap do L leu 248 MB do disco (uma vez, na primeira varredura) e cada recuperação levou 30,3 ms. Com 64 MB, menos que um terço da matriz, o kernel precisa descartar páginas de arquivo já usadas para trazer as próximas, e a varredura seguinte encontra o arquivo fora da memória: foram 20.132 faltas maiores, 8.537 MB lidos do disco e recuperação de 308,7 ms (10,2x). É o padrão de *thrashing* de páginas de arquivo: o processo continua correto, mas passa o tempo esperando o disco. Como as páginas são limpas, nada precisou ir para o swap.

read sob limite: falha. O `read()` precisa de cerca de 336 MB de memória anônima. Com 384 MB coube; com 192 MB e sem swap, as três execuções foram mortas pelo OOM killer do cgroup (SIGKILL, código -9) ainda na carga do índice. Com swap, a memória anônima excedente pode ir para o disco: com 256 MB, 3 de 3 concluíram, com pico de 96 MB em swap e recuperação de 49,8 ms contra 20,9 ms sem limite. Com 192 MB e swap, 2 de 3 concluíram e 1 foi morta, e as que concluíram ficaram 32x mais lentas: a cada varredura a matriz inteira volta do swap, e quando a recuperação de páginas não acompanha a demanda o OOM killer age mesmo com swap livre. O `vmstat` registrou picos de 361 MB/s em `si` e 254 MB/s em `so` nesses intervalos (results/evidence/vmstat_c5c.txt), a assinatura de swap intenso.

8.5 C6: falhas controladas

Tabela 12. C6: falhas injetadas e recuperação. Saída 2 = erro controlado com mensagem; -9 = morto pelo OOM killer.

Falha	Variante	Comportamento observado (média de 3 repetições)	Saída
Arquivo ausente (ingestão do S)	5 caminhos inexistentes	ingestão concluída com 500 documentos; 5 eventos falha_ingestao no log	0
Índice indisponível	vectors.f32 apagado	IndiceIndisponivel: arquivo do índice ausente; nenhuma requisição atendida	2
Índice indisponível	vectors.f32 truncado pela metade	inconsistente: 1.380.352 bytes, esperado 2.760.704; servidor não sobe	2
Índice indisponível	apagado + `--reconstruir-se-ausente`	reingestão automática e atendimento normal em 0,92 s	0
Fila saturada (1 trab., 4 prod., 80 req.)	fila 2, timeout 5 ms, sem retentativa	75 de 80 descartadas	0
Fila saturada	fila 2, timeout 5 ms, 3 retentativas	64 descartadas; 274 rejeições de put	0
Fila saturada	fila 64, timeout 30 s	0 descartadas; espera média na fila de 2,6 s	0
Limite de memória (L, cgroup 192 MB, sem swap)	índice por read()	3 de 3 mortas por OOM (SIGKILL)	-9
Limite de memória	índice por mmap()	3 de 3 concluídas	0
Serviço externo (100 req.)	falha 20%: 0 e 2 retentativas	erros de 22 para 1; serviço de 66 para 84 ms	0
Serviço externo	falha 50%: 0 e 2 retentativas	erros de 49 para 12; serviço de 66 para 105 ms	0
Ollama indisponível	porta sem serviço, 1 retentativa	10 de 10 requisições com erro controlado; trabalhadores seguem	0

Todas as falhas foram detectadas e registradas no log com evento próprio (`falha_ingestao`, `fila_cheia`, `requisicao_descartada`, `falha_geracao`). Arquivo ausente não interrompe a ingestão; índice ausente ou truncado é detectado antes de servir (verificação do tamanho esperado de `vectors.f32`) e, com `--reconstruir-se-ausente`, o servidor reconstrói o índice e atende normalmente. Fila saturada com timeout curto descarta requisições (backpressure explícito); retentativas com recuo exponencial reduzem os descartes ao custo de latência; fila grande elimina descartes e transforma o excesso em espera. Falhas do serviço externo viram erros por requisição sem derrubar trabalhadores; duas retentativas com recuo reduziram os erros de 22 para 1 (falha de 20%) e de 49 para 12

(50%), ao custo de tempo de serviço maior. O limite de memória é a única falha que mata o processo, e a recuperação é de projeto: carregar o índice por mmap.

8.6 Parte D: sincronização e falhas controladas

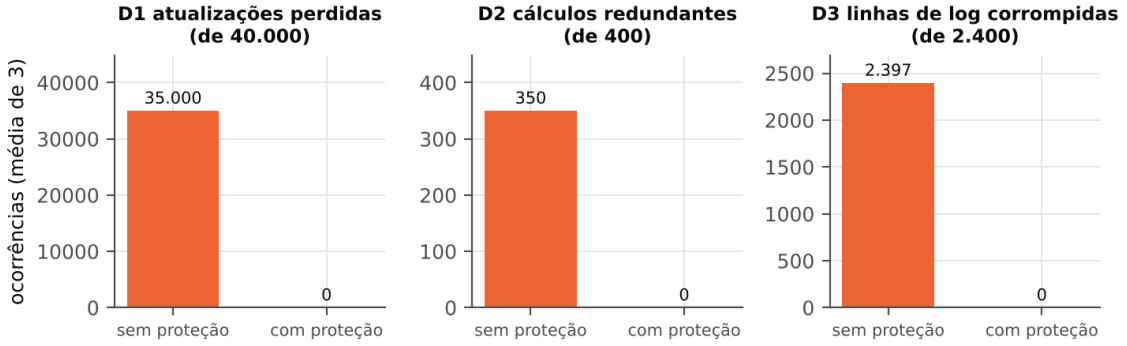


Figura 8. Falhas controladas sem proteção e com a sincronização adotada (média de 3 repetições).

Tabela 13. Parte D: falhas controladas e correção (média de 3 repetições).

Demonstração	Sem proteção (falha)	Com sincronização
D1 Contador compartilhado (8 threads × 5.000)	0 perdidas sem pausa; 35.000 com janela	0 perdidas; 3.011 ms contra 391 ms
D2 Cache, mesma chave (8 threads × 50 rodadas)	350 cálculos redundantes (none)	lock: 350; singleflight: 0
D2 Cache no servidor (carga repetida, 4 trab.)	none: 8,3 redundantes; contagem 111 de 120	lock: 7,0; singleflight: 0,0; contagem 120
D3 Log (8 threads × 300 linhas)	2.397 de 2.400 linhas corrompidas	0 corrompidas
D3 Log no servidor	28 de 123 linhas inválidas	0 inválidas
D4 Consumidor bloqueado (chegada lenta)	122 get() bloqueados; 1,8 s de espera somada	comportamento esperado do semáforo full
D4 Produtor bloqueado (fila 2, 1 trab.)	119 put() bloqueados; 15,3 s de espera somada	comportamento esperado do semáforo empty
D5 Espera circular (2 travas, ordens opostas)	ingênuo: travou nas 3 execuções, encerrado pelo pai após 3 s	timeout: concluiu em 1,94 s com retentativas; ordenado: 0,27 s, 0 retentativas
D6 Starvation (6 leitores, 1 escritor)	preferência a leitores: escritor esperou 1,45 s	preferência a escritor: 3,5 ms

Atualização perdida. O contador sem mutex não perdeu nada sem a pausa artificial: o GIL só troca de thread a cada 5 ms, e o laço termina antes. A corrida existe, mas é rara, o tipo de defeito que passa nos testes e aparece em produção. Com `sleep(0)` entre ler e gravar, 8 threads perderam 35.000 de 40.000 incrementos. Com mutex, nada se perde, mas a mesma janela dentro da região crítica tornou o laço 7,7x mais lento: quem cede a CPU segurando a trava faz as demais threads acordarem e dormirem de novo (efeito comboio). Regiões críticas devem ser curtas e não bloquear.

Cache e log no servidor. Sem mutex, os contadores do cache somaram 111 acertos+erros para 120 requisições, ou seja, atualizações perdidas num componente real, e houve 8,3 cálculos redundantes. O mutex corrigiu os contadores, mas não a duplicação (7,0), porque o cálculo fica fora da região crítica; só o singleflight a eliminou. No log sem mutex e gravado em fragmentos, 28 de 123 linhas ficaram inválidas (exemplo no Anexo A).

Deadlock. A versão ingênua travou nas três execuções; no momento do timeout, as duas threads estavam no estado S com canal de espera `futex_do_wait` (Anexo A), isto é, bloqueadas no kernel esperando uma trava que a outra segura: as quatro condições de Coffman presentes. O processo pai detectou a falta de progresso por timeout e encerrou o filho (SIGTERM, código -15), o mecanismo de encerramento exigido. A versão com `acquire(timeout)` quebra a condição de não preempção: detecta, libera o que tem e tenta de novo após espera aleatória, concluindo com cerca de 82 retentativas por thread. A versão ordenada quebra a espera circular e é a mais rápida (7,3x mais rápida que a com timeout).

Starvation. Com preferência a leitores e leitores sobrepostos, o escritor esperou 1,45 s (praticamente toda a execução) e completou só 1 de 5 escritas; com preferência a escritores, novos leitores esperam quando há escritor na fila, e a espera máxima caiu para 3,5 ms. No servidor, a situação análoga seria uma reindexação (escrita) que nunca consegue a trava porque consultas (leituras) não param de chegar.

9. Análise de memória, armazenamento, paginação e E/S

- **Corpus, índice, lote e contexto x RAM, paginação e latência.** Com `read`, a RAM cresce linearmente com o índice (40 para 326 MB de S para L) e a carga também (3,4 para 174 ms). A latência de recuperação cresce com o

número de trechos porque a busca é exata. Contexto maior toca mais páginas de texto e encarece a geração; lote maior eleva o pico de RSS da ingestão.

- **read x mmap.** read: custo proporcional ao arquivo inteiro, em tempo (cópia e faltas menores de zero-fill) e em memória anônima. mmap: custo proporcional ao que é tocado, com memória de arquivo compartilhável e descartável. No acesso esparso ao XL, 22 MB contra 1.030 MB; na varredura com cache quente, 80 contra 622 ms. Na primeira execução (cache frio) os dois são limitados pelo disco; na segunda, o mmap aproveita o cache de páginas sem cópia.
- **Evidência de paginação intensa e swap.** Sim, de forma controlada: mmap sob 64 MB gerou 20.132 faltas maiores e releu 8.537 MB; read sob 256 MB com swap usou até 96 MB de swap e ficou 2x mais lento; read sob 192 MB sem swap foi morto pelo OOM. Sem limite de cgroup, MemAvailable nunca ficou abaixo de 7 GB, e as faltas maiores foram nulas no C4 e no C5a, com o índice já no cache de páginas.

10. Análise de concorrência, cache e correção

- **O cache reduziu tempo ou operações?** Sim quando há repetição: 84% de acerto, 25 recuperações em vez de 150 e tempo de serviço 5,9x menor. Com consultas distintas, 0% e nenhum ganho.
- **A concorrência aumentou a vazão?** Para a carga de E/S, sim: 4,0x com 4 trabalhadores em 2 vCPUs. Para CPU em Python puro, o ganho foi limitado pelo GIL. A contenção aparece na fila: quando a chegada supera o serviço, a espera cresce com a capacidade da fila e o p95 explode, sem ganho de vazão.
- **Recursos que exigiram exclusão mútua ou sincronização de eventos.** Exclusão mútua: deque da fila, estrutura e contadores do cache, log, lista de resultados. Sincronização de eventos: semáforos `empty`/`full` (há vaga / há item), `Event` do singleflight (resultado pronto), pílula de parada para encerramento.
- **Falhas demonstradas.** Atualização perdida, cálculo duplicado, log intercalado, exceções e contadores incoerentes no cache, fila saturada com descarte, consumidor e produtor bloqueados, espera circular (deadlock) isolada e starvation de escritor.
- **Mecanismos e trade-offs.** Mutex: correção com custo baixo em regiões curtas, alto se a região bloqueia (comboio no D1). Semáforos com timeout: backpressure explícito, mas exigem política de retentativa e descarte. Singleflight: elimina trabalho repetido, ao custo de fazer threads esperarem pelo cálculo de outra (e de propagar a falha dela). Ordem global de travas: previne deadlock sem custo em tempo de execução, mas exige disciplina no código; timeout: detecta e recupera, mas desperdiça trabalho (7,3x mais lento que o ordenado).

11. Relação com Sistemas Operacionais e IA generativa

- **Pesos e índices são arquivos mapeados.** Runtimes como llama.cpp (base do Ollama) carregam pesos GGUF com `mmap`: o modelo "carrega" em milissegundos e as páginas entram sob demanda, exatamente como `vectors.f32` no C2. A diferença entre a primeira e a segunda execução da AV1 é o cache de páginas: na segunda, os pesos já estavam em RAM.
- **RSS de arquivo x anônima decide quem sobrevive à pressão.** Pesos mapeados e índices somente leitura podem ser descartados e relidos (C5c com mmap sob limite); buffers anônimos, KV-cache e tensores intermediários não, e terminam em swap ou OOM (read sob 192 MB). Por isso servidores de LLM pré-alocam e limitam o KV-cache, e o vLLM o pagina em blocos fixos (PagedAttention), aplicando ao KV-cache a mesma ideia de paginação da memória virtual.
- **Contexto e lote são memória.** Top-k maior aumentou bytes tocados, faltas de página e custo de prefill; o lote de 65.536 na ingestão multiplicou o pico de RSS por 2,6 em relação ao de 256, com ganho pequeno de vazão. Em LLMs, contexto e lote escalam o KV-cache e a ocupação de VRAM.
- **Checkpoints e embeddings em disco.** Gravar e reler checkpoints ou matrizes de embeddings é E/S sequencial grande: readahead ajuda, `MADV_RANDOM` destrói o desempenho, e o cache de páginas pode esconder o custo do disco em medições ingênuas (por isso as medidas frias com drop_caches).
- **Servir modelo é produtor/consumidor.** Requisições em fila limitada, workers, backpressure (rejeição com timeout), retentativa com recuo e cache de respostas (incluindo singleflight contra o efeito manada) são os mesmos mecanismos de servidores de inferência reais. O GIL do Python explica por que servidores de modelo fazem a computação pesada em C/C++ ou em processos separados.

12. Limitações

- Embeddings por hashing e geração simulada preservam o padrão de memória e espera, mas não a computação de um modelo real; o custo de CPU/GPU de um transformer não foi medido aqui.
- Um único nó com 2 vCPUs, sem GPU e sem VRAM: não há transferência PCIe, paginação de VRAM, NCCL nem contenção entre GPUs; múltiplas GPUs e ambientes distribuídos acrescentam rede, sincronização entre nós e falhas parciais que este recorte não cobre.
- Disco virtual (virtio) em VM de nuvem: latências e vazão não equivalem a um NVMe local; o cache do hipervisor pode suavizar leituras "frias".
- Python com GIL limita a concorrência de CPU; os resultados de escalabilidade valem para cargas dominadas por espera de E/S.
- Busca vetorial exata por varredura; índices aproximados (HNSW, IVF) tocam muito menos memória por consulta e mudariam o perfil de faltas de página.
- Corpus sintético e cargas determinísticas; tráfego real tem distribuição de popularidade e de tamanhos diferente. Três repetições bastam para os efeitos grandes medidos, não para diferenças pequenas.
- Os experimentos foram executados na VM descrita na Seção 3, e não no notebook do discente; a reexecução é feita por `run_all.sh`.

13. Conclusão

Respondendo à pergunta norteadora: **memória e armazenamento** definem o custo de ter o índice disponível, e a forma de leitura importa tanto quanto o tamanho. `read()` paga cópia e memória anônima proporcionais ao arquivo; `mmap()` paga faltas de página só no que é tocado e deixa o kernel administrar o resto como cache de páginas. Sob limite de memória, essa diferença separa degradação (mmap relendo do disco) de falha (OOM). **Filas e concorrência** aumentam a vazão quando a carga é dominada por espera e não quando é CPU sob o GIL; uma fila maior não cria capacidade, só converte bloqueio em latência. **Caches** economizam a maior parte do trabalho quando há repetição, nada quando não há, e exigem sincronização não só para não corromper a estrutura, mas também para não repetir trabalho. **Sincronização** é condição de correção: sem ela houve atualizações perdidas, logs corrompidos e cálculo duplicado; com ela, o custo foi pequeno nas regiões curtas e exigiu desenho (singleflight, ordem de travas, timeouts) nas longas.

14. Referências

ARPACI-DUSSEAU, R. H.; ARPACI-DUSSEAU, A. C. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. Disponível em: https://pages.cs.wisc.edu/~remzi/OSTEP/.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 10. ed. Wiley, 2018.

TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. 4. ed. Pearson, 2014.

GREGG, B. *Systems Performance: Enterprise and the Cloud*. 2. ed. Addison-Wesley, 2020.

THE LINUX KERNEL. The /proc filesystem; Memory Resource Controller (cgroup v1). Disponível em: https://docs.kernel.org/filesystems/proc.html e https://docs.kernel.org/admin-guide/cgroup-v1/memory.html.

LINUX MAN-PAGES PROJECT. mmap(2), madvise(2), mincore(2), posix_fadvise(2), read(2), write(2), getrusage(2), prctl(2), proc(5). Disponível em: https://man7.org/linux/man-pages/.

PYTHON SOFTWARE FOUNDATION. threading, mmap e sqlite3. *Python 3 Documentation*. Disponível em: https://docs.python.org/3/library/.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *NeurIPS*, 2020.

KWON, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *SOSP*, 2023.

WEINBERGER, K. et al. Feature Hashing for Large Scale Multitask Learning. In: *ICML*, 2009.

GGML-ORG. llama.cpp. Disponível em: https://github.com/ggml-org/llama.cpp. OLLAMA. Disponível em: https://github.com/ollama/ollama.

Anexo A. Evidências do sistema operacional

Trechos de `results/evidence/` (arquivos completos no repositório).

Processo em execução: threads, memória e mapeamentos (corpus L, 4 trabalhadores)

```
### ps -o pid,ppid,nlwp,stat,rss,vsz,pcpu,cmd -p 7961
  PID  PPID NLWP STAT   RSS    VSZ %CPU CMD
 7961  7894    9 Sl   311192 1099476 151 python3 -m genresearch serve --indice index/L --saida results/raw/evid
--rotu
### ps -L (threads)
  PID   LWP STAT %CPU COMMAND         WCHAN
 7961  7961 Sl    4.4 python3         futex_do_wait
 7961  7963 Rl   91.8 python3         -
 7961  7964 Sl    0.4 amostrador      futex_do_wait
 7961  7965 Sl   15.1 worker-0        hrtimer_nanosleep
 7961  7968 Sl   14.2 worker-1        hrtimer_nanosleep
 7961  7969 Sl   13.9 worker-2        hrtimer_nanosleep
 7961  7970 Sl   13.5 worker-3        hrtimer_nanosleep
 7961  7971 Sl    0.0 produtor-0      futex_do_wait
 7961  7972 Sl    0.0 produtor-1      futex_do_wait
### /proc/7961/task/*/comm
7961 python3 futex_do_wait
7963 python3 0
7964 amostrador futex_do_wait
7965 worker-0 hrtimer_nanosleep
7968 worker-1 0
7969 worker-2 hrtimer_nanosleep
7970 worker-3 hrtimer_nanosleep
7971 produtor-0 futex_do_wait
7972 produtor-1 futex_do_wait
### /proc/7961/status (memoria)
Name:python3
State:S (sleeping)
VmPeak: 1104380 kB
VmSize: 1099476 kB
VmHWM:  312424 kB
VmRSS:  312424 kB
RssAnon:   39664 kB
RssFile:  272760 kB
RssShmem:       0 kB
```

strace -f -c de uma execução completa do servidor (M, 4 trabalhadores, 80 requisições)

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 66.78    0.183446          93      1968       151 futex
 19.47    0.053493         906        59           clock_nanosleep
  2.57    0.007071         128        55           madvise
  2.49    0.006839          17       393           close
  2.47    0.006779           7       959        95 newfstatat
  1.97    0.005411           9       601           read
  1.40    0.003846          25       150           mprotect
  0.88    0.002430           4       590           pread64
  0.42    0.001166           1       664           fstat
  0.37    0.001020           3       259           mmap
  0.33    0.000906           2       385        15 openat
  0.24    0.000652           1       482        12 lseek
  0.12    0.000323           1       317       317 ioctl
  0.08    0.000210           2        94           brk
  0.06    0.000176           1       121           rt_sigaction
  0.05    0.000151           8        18           geteuid
```

strace -e: read() sobre vectors.f32 (L)

```
7920  15:46:36.171641 openat(AT_FDCWD, "index/L/vectors.f32", O_RDONLY|O_CLOEXEC) = 3
7920  15:46:36.171707 read(3, "...", 222569473) = 222569472
7920  15:46:36.302444 read(3, "", 1)    = 0
7920  15:46:36.361112 close(3)          = 0
--- read() sobre o fd 3: 2 chamadas, 222569472 bytes
```

strace -e: mmap() sobre vectors.f32 (L)

```
7936  15:46:36.562064 openat(AT_FDCWD, "index/L/vectors.f32", O_RDONLY|O_CLOEXEC) = 3
7936  15:46:36.562120 mmap(NULL, 222569472, PROT_READ, MAP_SHARED, 3, 0) = 0x7fc62dc00000
7936  15:46:36.600392 close(3)          = 0
--- read() sobre o fd 3: 0 chamadas, 0 bytes
```

Deadlock ingênuo: estado das threads no momento do timeout

```
tid=5030 comm=python3 estado=S wchan=futex_do_wait
tid=5031 comm=dl-t0 estado=S wchan=futex_do_wait
tid=5032 comm=dl-t1 estado=S wchan=futex_do_wait
```

Exemplo de linha de log corrompida (modo sem mutex)

```
{"ts": 179252190{"ts": 179252190{"ts": 179252190{"ts": 179252190{"ts": 179252190{"ts": 1792521906.857121,
"threa6.857227
```

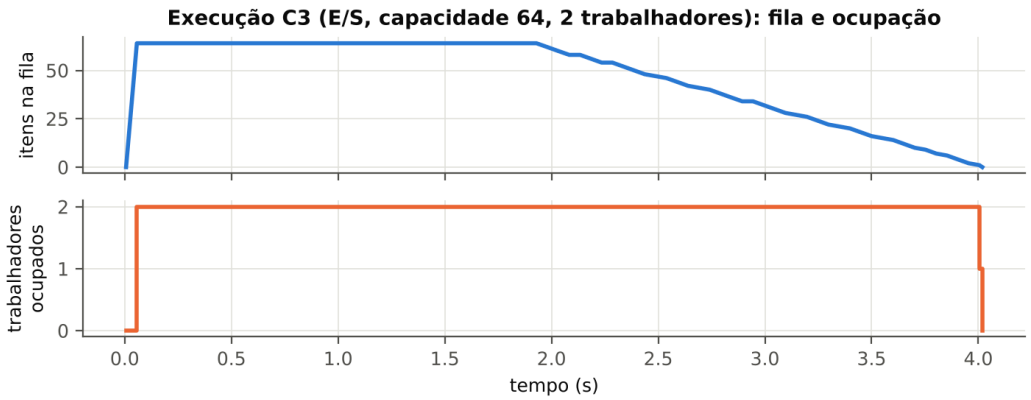


Figura A1. Série temporal de uma execução do C3 (E/S, fila 64, 2 trabalhadores): a fila enche na rajada e é drenada a taxa constante.